

05-1 파이썬 [클래스]

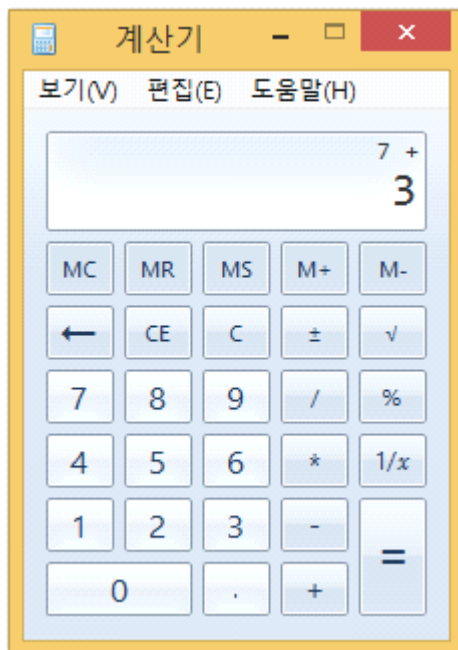
클래스는 도대체 왜 필요한가?

가장 많이 사용하는 프로그래밍 언어 중 하나인 C 언어에는 클래스가 없다. 이 말은 굳이 클래스 없이도 프로그램을 충분히 만들 수 있다는 말과도 같다. 파이썬으로 잘 만들어진 프로그램들을 살펴봐도 클래스를 이용하지 않고 작성된 것들이 상당히 많다. 클래스는 지금까지 공부한 함수나 자료형처럼 프로그램 작성을 위해 꼭 필요한 요소는 아니다.

하지만 프로그램 작성시 클래스를 적재적소에 이용하면 프로그래머가 얻을 수 있는 이익은 상당하다.

예제를 통해 한번 생각해 보자.

여러분 모두 계산기를 사용해 봤을 것이다. 계산기에 3이라는 숫자를 입력하고 + 기호를 입력한 후 4를 입력하면 결과값으로 7을 보여준다. 다시 한 번 + 기호를 입력한 후 3을 입력하면 기존 결과값 7에 3을 더해 10을 보여준다. 즉, 계산기는 이전에 계산된 결과값을 항상 메모리 어딘가에 저장하고 있어야 한다.



(※ 계산기는 이전에 계산된 결과값을 기억하고 있어야 한다.)

이런 내용을 우리가 앞서 익힌 함수를 이용해 구현해 보자. 계산기의 "더하기" 기능을 구현한 파이썬 코드는 다음과 같다.

```
result = 0
```

```
def adder(num):  
    global result  
    result += num  
    return result
```

```
print(adder(3))
print(adder(4))
```

(※ adder 함수는 입력 인수로 num을 받으면 이전에 계산된 결과값에 더한 후 출력하는 함수이다.)

이전에 계산된 결과값을 유지하기 위해서 result라는 전역 변수(global)를 사용했다. 실행하면 예상한 대로 다음과 같은 결과값이 출력된다.

```
3
7
```

그런데 만약 한 프로그램에서 2개의 계산기가 필요한 상황이 발생하면 어떻게 해야 할까? 각각의 계산기는 각각의 결과값을 유지해야 하기 때문에 위와 같이 adder 함수 하나만으로는 결과값을 따로 유지할 수 없다.

이런 상황을 해결하려면 다음과 같이 함수를 각각 따로 만들어야 한다.

```
result1 = 0
result2 = 0
```

```
def adder1(num):
    global result1
    result1 += num
    return result1
```

```
def adder2(num):
    global result2
    result2 += num
    return result2
```

```
print(adder1(3))
print(adder1(4))
print(adder2(3))
print(adder2(7))
```

똑같은 일을 하는 adder1과 adder2라는 함수가 만들어졌고 각각의 함수에서 계산된 결과값을 유지하면서 저장하기 위한 전역 변수 result1, result2가 필요하게 되었다.

결과값은 다음과 같이 의도한 대로 출력된다.

```
3
7
3
10
```

계산기 1의 결과값이 계산기 2에 아무런 영향을 끼치지 않음을 확인할 수 있다. 하지만 계산기가 3개, 5개, 10개로 점점 더 많이 필요해진다면 어떻게 해야 할 것인가? 그때마다 전역변수와 함수를 추가할 것인가? 여기에 빼기나 곱하기등의 기능을 추가해야 한다면 상황은 점점 더 어려워질 것이다.

아직 클래스에 대해서 배우진 않았지만 위와 같은 경우 클래스를 이용하면 다음과 같이 간단하게 해결할 수 있다.

(※ 아래 예시 클래스를 아직은 이해하지 못해도 좋다. 곧 자세하게 배울 것이다. 여기서는 클래스를 개념적으로만 이해하면 된다.)

```
class Calculator:
    def __init__(self):
        self.result = 0

    def adder(self, num):
        self.result += num
        return self.result
```

```
cal1 = Calculator()
cal2 = Calculator()
```

```
print(cal1.adder(3))
print(cal1.adder(4))
print(cal2.adder(3))
print(cal2.adder(7))
```

실행하면 함수 2개를 사용했을 때와 동일한 결과가 출력된다.

```
3
7
3
10
```

Calculator 클래스로 만들어진 cal1, cal2라는 별개의 계산기(파이썬에서는 이것을 객체라고 한다)가 각각의 역할을 수행한다. 그리고 계산기(cal1, cal2)의 결과값 역시 다른 계산기의 결과값과 상관없이 독립적인 결과값을 유지한다. 클래스를 이용하면 계산기의 개수가 늘어나더라도 객체를 생성하기만 하면 되기 때문에 함수를 사용하는 경우와 달리 매우 간단해진다.

만약 빼기 기능이 더해진다고 해도 Calculator 클래스에 다음과 같은 빼기 기능 함수를 추가해 주면 된다.

```
def sub(self, num):
    self.result -= num
    return self.result
```

클래스의 이점은 단순히 이것만이 아니다. 하지만 이것 하나만으로도 "도대체 왜 클래스가 필요한 것일까?"라는 근본적인 물음에 대한 해답이 되었을 것이다.

클래스와 객체

클래스를 가장 잘 설명 해 주는 다음의 사진을 보자.



과자를 만드는 과자틀과 만들어진 과자들이다.

- 과자틀 → 클래스 (class)
- 과자틀에 의해서 만들어진 과자들 → 객체 (object)

이 사진을 보면 클래스가 어떤건지 감이 잡힐 것이다.

이 절에서 설명할 클래스는 과자틀과 비슷하다.

클래스(class)란 똑같은 무엇인가를 계속해서 만들어낼 수 있는 설계 도면 같은 것이고(과자 틀), 객체(object)란 클래스에 의해서 만들어진 피조물(과자틀에 의해서 만들어진 과자)을 뜻한다.

클래스에 의해서 만들어진 객체에는 중요한 특징이 있다. 그것은 객체별로 독립적인 성격을 갖는다는 것이다. 과자틀에 의해서 만들어진 과자에 구멍을 뚫거나 조금 베어먹더라도 다른 과자들에는 아무 영향이 없는 것과 마찬가지로 동일한 클래스에 의해 생성된 객체들은 서로에게 전혀 영향을 주지 않는다.

다음은 파이썬 클래스의 가장 간단한 예이다.

```
>>> class Programmer:
>>>     pass
```

위의 클래스는 아무런 기능도 갖고 있지 않은 껍질뿐인 클래스이다. 하지만 이렇게 껍질뿐인 클래스도 객체를 생성하는 기능은 가지고 있다. "과자 틀"로 "과자"를 만드는 것처럼 말이다.

객체는 클래스에 의해서 만들어지며 1개의 클래스는 무수히 많은 객체를 만들어낼 수 있다. 위에서 만든 Programmer 클래스의 객체를 만드는 방법은 다음과 같다.

```
>>> kim = Programmer()
>>> park = Programmer()
```

Programmer()의 결과값을 돌려받은 kim과 park이 바로 객체이다. 마치 함수를 사용해서 그 결과값을 돌려받는 모습과 비슷하다.

[객체와 인스턴스의 차이]

클래스에 의해서 만들어진 객체를 인스턴스라고도 한다. 그렇다면 객체와 인스턴스의 차이는 무엇일까? 이렇게 생각해 보자. `kim = Programmer()` 이렇게 만들어진 `kim`은 객체이다. 그리고 `kim`이라는 객체는 `Programmer`의 인스턴스이다. 즉, 인스턴스라는 말은 특정 객체(`kim`)가 어떤 클래스(`Programmer`)의 객체인지를 관계 위주로 설명할 때 사용된다. 즉, "`kim`은 인스턴스" 보다는 "`kim`은 객체"라는 표현이 어울리며, "`kim`은 `Programmer`의 객체" 보다는 "`kim`은 `Programmer`의 인스턴스"라는 표현이 훨씬 잘 어울린다.

이야기 형식으로 클래스 기초 쌓기

지금부터는 클래스의 개념을 아직 제대로 이해하지 못한 독자들을 위해 설명한다. 아주 쉽게 설명하려고 노력하였고 최대한 클래스의 핵심 개념에 가까이 다가갈 수 있도록 배려하였다.

클래스 변수

다음의 클래스를 보자.

```
>>> class Service:
...     secret = "영구는 외계인이다."
```

위의 클래스 이름은 `Service`이다. 우리는 이 `Service` 클래스를 어떤 유용한 정보를 제공해 주는 한 인터넷 서비스 제공 업체라고 가정하자. 이 인터넷 업체는 "`영구는 외계인이다.`"라는 비밀 정보를 가지고 있다. 먼저 인터넷 업체에 가입을 해 보자.

가입하는 방법은 다음과 같다.

```
>>> pey = Service()
```

위와 같이 하면 `pey`라는 아이디(객체)로 인터넷 서비스 업체인 `Service` 클래스를 이용할 수 있다.

이제 `pey`라는 아이디로 위 서비스 업체가 제공하는 정보를 얻어내자.

```
>>> pey.secret
"영구는 외계인이다."
```

아이디 이름에 서비스 업체가 제공하는 `secret`이라는 변수를 `'.'`(도트 연산자)를 이용해서 호출하였더니 실로 어마어마한 정보를 얻을 수 있었다.

알아두기

클래스 변수는 객체간 서로 공유되는 변수로 보통 클래스에 의해 생성되는 객체들이 공통적으로 사용할 목적으로 쓰인다. 따라서 `Service`클래스에 의해 만들어진 객체는 모두 `secret`이라는 클래스 변수를 사용할 수 있게 되는 것이다.

클래스 변수는 객체를 통한 접근(예:pey.secret)방법 외에 클래스를 통한 접근도 가능하다. 클래스를 통해 직접 클래스변수에 접근하는 방법은 다음과 같이 **클래스명.클래스변수** 와 같이 호출하여 사용할 수 있다.

```
>>> Service.secret
"영구는 외계인이다."
```

클래스 변수는 "공유되는 변수" 라는 특징을 가지고 있다고 했다. 이 사실을 확인해 보기 위해 다음과 같이 클래스 변수의 값을 변경해 보자.

```
>>>Service.secret = "영구는 도깨비이다."
```

클래스 변수가 공유되는 변수라면 이미 생성한 pey객체의 secret 값도 역시 변경되었을 것이다.

다음과 같이 확인해 보자.

```
>>> print(pey.secret)
영구는 도깨비이다.
```

pey 객체가 참조하는 secret 값이 변경되었음을 확인할 수 있다.

클래스 내부의 함수(메서드)

Service라는 이름의 이 인터넷 서비스 제공 업체는 생긴 지 얼마 되지 않은 벤처 기업이라서 이와 같이 단 한 가지 정보만을 제공하고 있었다. 제공해 주는 정보의 양이 너무 적은 듯하여 가입한 사람들을 대상으로 어떤 정보를 더 제공하면 좋을지 설문 조사를 하였더니 모두들 더하기를 잘하지 못한다고 대답했다. 그래서 이 서비스 업체는 더하기를 해주는 서비스를 제공하기로 마음을 먹었다.

두 수를 더하는 서비스를 제공해 주기 위해서 이 서비스 업체는 다음과 같이 클래스를 업그레이드했다.

```
>>> class Service:
...     secret = "영구는 외계인이다." # 유용한 정보
...     def sum(self, a, b):          # 더하기 서비스
...         result = a + b
...         print("%s + %s = %s입니다." % (a, b, result))
...
>>>
```

클래스가 업그레이드되어 이제 업체에 가입한 모든 사람들이 더하기 서비스를 제공받을 수 있게 되었다. 이 업체의 서비스를 이용하는 방법은 다음과 같다.

회원으로 가입되어 있지 않다면 먼저 서비스 업체에 가입을 해서 아이디를 받는다.

```
>>> pey = Service()
```

다음에 더하기 서비스를 이용한다. secret변수를 호출할 때와 마찬가지로 아이디에 도트 연산자를 이용하여 sum함수를 호출하도록 한다.

```
>>> pey.sum(1,1)
1 + 1 = 2입니다.
```

더하기의 결과값이 정상적으로 출력되었다.

self 살펴보기

이번에는 Service클래스에 추가한 더하기 함수(sum)를 자세히 살펴보자.

```
... def sum(self, a, b):
...     result = a + b
...     print("%s + %s = %s입니다." % (a, b, result))
```

sum함수에 뭔가 생소한 self라는 변수가 보일 것이다. 이것에 대해서 이해하는 것은 무척 중요하니 세심하게 살펴보도록 하자.

어떤 사람이 다음처럼 더하기 서비스를 이용하려 한다고 생각해 보자.

```
>>> pey = Service()
>>> pey.sum(1, 1)
```

이렇게 하면 pey라는 객체를 통해 Service클래스의 sum함수가 호출될 것이다.

그런데 잠깐 뭔가 좀 이상하지 않은가?

Service클래스의 sum 함수는 다음처럼 총 3개의 입력을 필요로 하는 함수이다.

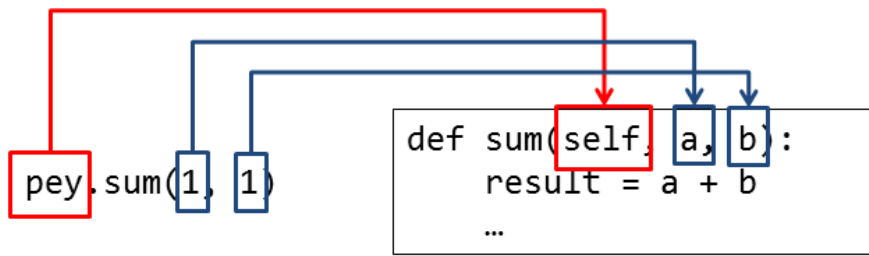
- 첫번째 인수 : self
- 두번째 인수 : a
- 세번째 인수 : b

하지만 실제 sum함수의 호출은 다음처럼 총 2개의 입력만으로 가능했다.

```
pey.sum(1, 1) # 첫번째 입력 1, 두번째 입력 1 로 sum 함수호출
```

왜 그럴까?

그 이유는 아래 그림처럼 `pey.sum(1, 1)`이라는 호출이 발생할 때 sum함수의 첫번째 인수인 self에는 호출 시 이용했던 객체(pey 객체)가 자동으로 전달되기 때문이다.



(참고. 이것은 파이썬만의 규칙이기 때문에 외우는 수밖에 없다. 다른 객체지향 언어들에서는 클래스 함수에 첫번째 파라미터로 객체를 직접 전달하는 특징이 없다.)

다음과 같은 파이썬의 특징을 꼭 기억해 두도록 하자.

"파이썬은 객체를 통해 클래스의 함수를 호출할 때 호출한 객체 자신이 호출한 클래스 함수의 첫 번째 입력 인수로 전달된다."

알아두기

Service클래스의 sum함수를 호출할 수 있는 방법은 한가지가 더 있다. 이전에 알아보았던 방법은 객체를 통한 호출이고 이번에 알아볼 방법은 클래스를 통한 호출이다.

클래스를 통해 sum함수를 호출하는 방법은 다음과 같다.

```
>>> pey = Service()
>>> Service.sum(pey, 1, 1)
```

객체를 통한 호출이 아니므로 sum함수에는 pey객체가 자동으로 전달되지 않는다. 따라서 클래스를 통해 sum함수를 호출 할 때는 위의 예와 같이 총 3개의 입력을 모두 넘겨주어야만 한다.

두 가지 방법의 호출 결과는 모두 동일하다.

객체를 통한 호출 방법이 일반적인 방법이며 클래스를 통한 호출은 특별한 경우를 제외하고는 잘 사용되지 않는다.

클래스 함수는 우리가 앞서 공부했던 함수가 클래스내에 포함된 형태이다. 함수가 클래스내에 종속되어져 있다는 점만 제외하면 기존 함수와 크게 다를 바가 없다.

클래스 함수는 다른말로 메서드(Method)라고 부른다.

파이썬 메서드의 첫번째 파라미터명은 관례적으로 self라는 이름을 사용한다. 호출 시 호출한 객체 자신이 전달되기 때문에 self("self"는 자기자신이라는 뜻을 가진 영어단어이다.)라는 이름을 사용하게 된 것이다. 물론 self 말고 다른 이름을 사용해도 상관은 없다.

(참고. 메서드의 첫번째 파라미터 self를 명시적으로 구현해야 하는 것은 파이썬만의 독특한 특징이다. 예를들어 자바같은 언어는 첫번째 파라미터인 self가 필요없다.)

객체 변수

시간이 흘러 더하기 서비스를 계속 제공받던 사람들은 서비스 업체에 뭔가 색다른 서비스를 요구하기 시작했다. 그 요구는 더하기 서비스를 제공할 때 "홍길동 님 $1 + 2 = 3$ 입니다." 처럼 홍길동이라는 자신의 이름을 넣어 달라는 것이었다.

인터넷 서비스 업체인 Service는 "이름 출력 서비스"를 제공해 주기로 마음을 먹었다. 그래서 다음과 같이 또 Service 클래스를 업그레이드하게 되었다.

사용자에게 이름을 입력받아서 더하기 서비스를 제공할 때 앞부분에 그 이름을 넣어 주도록 다음과 같이 변경하였다.

```
>>> class Service:
...     secret = "영구는 외계인이다."
...     def setname(self, name):
...         self.name = name
...     def sum(self, a, b):
...         result = a + b
...         print("%s님 %s + %s = %s입니다." % (self.name, a, b, result))
...
>>>
```

그리고 바뀐 점과 "이름 출력 서비스"의 사용법에 대해서 가입한 사람들에게 알려주었다. 그래서 사람들은 다음처럼 서비스를 이용할 수 있게 되었다.

먼저 서비스 업체에 가입을 해서 pey라는 아이디를 얻는다.

```
>>> pey = Service()
```

다음으로 pey라는 아이디를 가진 사람이 자신의 이름은 홍길동임을 서비스 업체에 알려준다.

```
>>> pey.setname("홍길동")
```

더하기 서비스를 이용한다.

```
>>> pey.sum(1, 1)
홍길동님 1 + 1 = 2입니다.
```

이제 서비스 업체의 입장에서 다시 한 번 생각해 보자.

아래처럼 pey라는 아이디를 부여받은 사람이 있다.

```
>>> pey = Service()
```

이 사람이 자신의 이름을 설정하겠다는 요구를 다음과 같이 했다.

```
>>> pey.setname("홍길동")
```

setname메서드가 실행되는 순서를 보다 자세하게 살펴보도록 하자.

1. pey라는 아이디를 가진 사람이 "홍길동"이라는 이름을 setname 메서드에 입력으로 준다.

```
>>> pey.setname("홍길동")
```

2. 그러면 setname메서드의 다음 문장이 수행된다.

```
self.name = name
```

3. self는 setname 메서드의 첫 번째 입력값으로 pey라는 객체가 자동으로 전달되므로 다음과 같이 바뀔 것이다.

```
pey.name = name
```

4. name은 setname 메서드의 두 번째로 입력받은 "홍길동"이라는 값이므로 위의 문장은 다시 다음과 같이 바뀔 것이다.

```
pey.name = "홍길동"
```

객체변수는 다음과 같이 생성할 수 있다. (객체변수란 객체만의 변수를 의미한다.)

```
객체.객체변수 = 값
```

따라서 위 처럼 `pey.name = "홍길동"`이라는 문장이 수행되면 객체 pey에 객체변수 name이 "홍길동"이라는 값으로 저장되게 된다.

(※ 객체변수는 속성, 멤버변수 또는 인스턴스 변수라고 표현하기도 한다.)

알아두기

클래스 변수가 객체간 서로 공유되는 변수라면 객체 변수는 객체별로 고유한 값이 저장되는 변수이다.

이해를 돕기 위해 다음의 예제를 보자.

```
>>> kim = Service()
>>> park = Service()
>>> kim.name = "김철수"
>>> park.name = "박응용"
```

kim객체와 park객체에 name이라는 객체변수를 생성하고 그 값을 각각 "김철수"와 "박응용"으로 대입하였다.

그리고 다음과 같이 객체변수값을 출력해 보자.

```
>>> print(kim.name)
김철수
>>> print(park.name)
박응용
```

여기서 주목해야 할 점은 kim객체의 name에 "김철수"라는 값을 저장하고 park객체의 name에 "박응용"이라는 값을 저장하더라도 kim객체의 name값이 변하지 않는다는 점이다. 즉, 객체 변수의 값은 객체별로 고유하게 유지된다는 점을 꼭 기억하도록 하자.

다음으로 "홍길동님 1 + 1 = 2 입니다." 라는 서비스를 제공하기 위해 변경된 더하기 메서드를 보자.

```
... def sum(self, a, b):
...     result = a + b
...     print("%s님 %s + %s = %s입니다." % (self.name, a, b, result))
```

"1 + 1 = 2 입니다."라는 서비스만 제공했던 이전의 sum 메서드와 비교해 보면 다른 점은 단 하나, self.name이라는 것을 출력 문자열에 삽입한 것뿐이다. 그렇다면 self.name은 무엇일까?

먼저 `pey.setname("홍길동")` 문장이 수행되어 `pey.name`이라는 객체 변수에는 "홍길동"이라는 값이 저장된다는 사실을 이미 알아보았다. 따라서 `pey.sum(1, 1)` 과 같이 호출되면 sum 메서드에서도 첫번째 입력인수 self는 호출한 객체 pey가 자동으로 전달되므로 `self.name`은 `pey.name`으로 치환될 것이다. 이러한 이유로 sum메서드는 "홍길동"이라는 pey객체의 name객체 변수값을 출력할 수 있게 되는 것이다.

__init__ 이란 무엇인가?

자, 이제 또 시간이 흘러 가입자수가 많아지게 되었다. 서비스 업체가 아주 친절하게 더하기 서비스를 제공해 주었기 때문이다. 그런데 가끔 이상한 오류가 발생한다고 항의 전화가 빗발치듯 쏟아진다. 그 사람들의 말을 들어보면 다음과 같았다.

```
>>> babo = Service()
>>> babo.sum(1, 1)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 7, in sum
AttributeError: 'Service' object has no attribute 'name'
```

위와 같이 입력하면 자꾸 오류가 발생한다는 것이다. 그 때마다 서비스 업체에서는 `babo.setname("나바보")`와 같은, 이름을 넣어 주는 과정이 빠졌기 때문에 오류가 발생한다는 것이라고 골백 번 얘기를 하지만 항상 이런 실수를 하는 사람들로부터 항의 전화가 와서 여간 귀찮은 게 아니었다.

그래서 다음과 같은 아이디어를 생각해냈다. 지금까지는 사람들이 서비스에 가입할 때 별다른 과정 없이 바로 아이디를 부여해 주는 방식이었다. 그런데 만약 아이디를 부여해 줄 때 그 사람의 이름을 입력받아야만 아이디를 부여해 주는 방식으로 바꾸면 `babo.setname("나바보")`와 같은 과정을 생략할 수 있을 것이란 생각이었다.

위와 같이 서비스하기 위한 방법을 찾던 중 서비스 업체의 실력자 한 사람이 `__init__`이라는 메서드를 이용하자고 제의를 했고 그 방법은 다음과 같았다.

```
>>> class Service:
...     secret = "영구는 외계인이다."
...     def __init__(self, name):
...         self.name = name
...     def sum(self, a, b):
...         result = a + b
...         print("%s님 %s + %s = %s입니다." % (self.name, a, b, result))
...
>>>
```

위의 Service 클래스를 이전의 클래스와 비교해 보면 바뀐 부분은 딱 한 가지이다. 바로 setname 메서드의 이름인 setname이 `__init__`으로 바뀐 것이다. 클래스에서 이 `__init__`이라는 메서드는 특별한 의미를 갖는다. 그 의미는 다음과 같다.

"객체를 만들 때 항상 실행된다."

즉, 아이디를 부여받을 때 항상 실행된다는 말이다. 따라서 이제는 이 서비스 업체에 가입하려면 다음처럼 입력해야 한다.

```
>>> pey = Service("홍길동")
```

이전에는 `pey = Service()`라고만 입력하면 되었지만 이제는 `__init__` 메서드 때문에 `pey = Service("홍길동")`처럼 아이디를 부여받을 때 이름까지 함께 입력해야만 한다.

`__init__` 메서드는 setname 메서드와 동일한 방법으로 객체변수 name에 그 값을 설정한다. setname 메서드와 다른 점은 객체가 생성될 때 실행된다는 점 뿐이다.

이전에 서비스를 이용하기 위해 사용했던 방식은 다음과 같다.

```
>>> pey = Service()
>>> pey.setname("홍길동")
>>> pey.sum(1, 1)
```

위의 코드를 `__init__` 메서드를 이용하면 다음처럼 간략하게 작성할 수 있다.

```
>>> pey = Service("홍길동")
>>> pey.sum(1, 1)
```

서비스 방법을 이렇게 변경하고 난 후 빗발치던 항의 전화도 멈추게 되었고 세 번 입력하던 것을 두 번만 입력하면 되니 모두들 기뻐하였다.

지금까지 객체와 self, `__init__` 메서드의 의미를 설명하기 위해 이야기 형식으로 클래스에 대해서 알아보았다. 위의 내용은 비록 클래스에 대한 정확한 설명은 아니지만 초보자가 객체와 self의 의미, 그리고 `__init__` 메서드에 대해서 조금 더 쉽게 접근할 수 있었을 것이다. 위에서 살펴본 `pey = Service()`로 해서 생성된 pey를 아이디라고 하였는데, 이것이 바로 객체라고 불린다는 것을 잊지 말자.

이제 위에서 살펴보았던 기초적인 사항을 바탕으로 클래스에 대해서 자세하게 알아보자.

클래스 자세히 알기

클래스란 객체를 만들어내는 공장과도 같다. 이 객체를 어떻게 사용할 수 있는지를 알려면 클래스의 구조를 보면 된다. 즉, 클래스는 해당 객체의 청사진(설계도)이라고 할 수 있다.

이제부터 파이썬 프로그래밍의 가장 중심이 되는 클래스에 대해서 자세하게 알아보기로 하자. 잘 이해가 되지 않더라도 낙심하지는 말자. 파이썬에 익숙해지면 반드시 쉽게 이해할 수 있게 될 것이다. 여러 가지 클래스를 만들어 보면서 클래스에 대해 자세하게 알아보자.

클래스의 구조

클래스는 기본적으로 다음과 같은 구조를 가지고 있다.

```
class 클래스이름[(상속 클래스명]):  
    <클래스 변수 1>  
    <클래스 변수 2>  
    ...  
    def 메서드1(self[, 인수1, 인수2,...]):  
        <수행할 문장 1>  
        <수행할 문장 2>  
        ...  
    def 메서드2(self[, 인수1, 인수2,...]):  
        <수행할 문장1>  
        <수행할 문장2>  
        ...  
    ...
```

위에서 봤듯이 class라는 키워드는 클래스를 만들 때 사용되는 예약어이고 그 바로 뒤에 클래스 이름을 입력한다. 클래스 이름 뒤에 상속할 클래스가 있다면 괄호() 안에 상속할 클래스 이름을 입력한다. 클래스 내부에는 클래스 변수와 메서드들이 있다.

클래스가 무엇인지 아직 감이 오지 않더라도 걱정하지 말고 다음의 예를 보며 차근차근 이해해보자.

사칙연산 클래스 만들기

이번에는 사칙연산을 해 주는 클래스를 작성해 보도록 하자. 사칙연산은 더하기, 빼기, 나누기, 곱하기를 말한다.

클래스를 어떻게 만들지 먼저 구상하기

클래스는 무작정 만들기 보다는 클래스에 의해서 만들어진 객체를 중심으로 어떤 식으로 동작하게 할 것인지 미리 구상을 한 후에 생각했던 것들을 하나씩 해결하면서 완성해 나가는 것이 좋다.

사칙연산을 가능하게 하는 FourCal이라는 클래스가 다음처럼 동작한다고 가정해 보자.

먼저 `a = FourCal()`처럼 입력해서 `a`라는 객체를 만든다.

```
>>> a = FourCal()
```

그런 다음 `a.setdata(4, 2)`처럼 입력해서 4와 2라는 숫자를 `a`에 지정해 주고

```
>>> a.setdata(4, 2)
```

`a.sum()`을 수행하면 두 수를 합한 결과(`4 + 2`)를 돌려주고

```
>>> print(a.sum())
6
```

`a.mul()`을 수행하면 두 수를 곱한 결과(`4 * 2`)를 돌려주고

```
>>> print(a.mul())
8
```

`a.sub()`를 수행하면 두 수를 뺀 결과(`4 - 2`)를 돌려주고

```
>>> print(a.sub())
2
```

`a.div()`를 수행하면 두 수를 나눈 결과(`4 / 2`)를 돌려준다.

```
>>> print(a.div())
2
```

이렇게 동작하는 FourCal 클래스를 만드는 것이 바로 우리의 목표이다.

클래스 구조 만들기

자, 그렇다면 지금부터는 앞에서 구상했던 것처럼 동작하는 클래스를 만들어 보자. 제일 먼저 할 일은 `a = FourCal()`처럼 객체를 만들 수 있게 하는 것이다. 일단은 아무 기능이 없어도 되기 때문에 만드는 것은 매우 간단하다. 다음을 따라 해보자.

```
>>> class FourCal:
...     pass
...
>>>
```

우선 대화형 인터프리터에서 `pass`란 문장만을 포함한 FourCal 클래스를 만든다. 현재 상태에서 FourCal 클래스는 아무런 변수나 메서드도 포함하지 않지만 우리가 원하는 객체 `a`를 만들 수 있는 기능은 가지고 있다. 확인해 보자.

(※ pass는 아무것도 수행하지 않는 문법이다. 임시로 코드를 작성할 때 주로 사용한다.)

```
>>> a = FourCal()
>>> type(a)
<class '__main__.FourCal'>
```

위와 같이 a = FourCal()로 a라는 객체를 먼저 만들고 그 다음에 type(a)로 a라는 객체가 어떤 타입인지 알아보았다. 역시 객체 a가 FourCal 클래스의 인스턴스임을 알 수 있다.

(※ type 함수는 파이썬이 자체적으로 가지고 있는 내장 함수로 객체의 타입을 출력한다.)

객체에 숫자 지정할 수 있게 만들기

하지만 생성된 객체 a는 아직 아무런 기능도 하지 못한다. 이제 더하기, 나누기, 곱하기, 빼기 등의 기능을 하는 객체를 만들어야 한다. 그런데 이러한 기능을 갖춘 객체를 만들려면 우선적으로 a라는 객체에 사칙연산을 할 때 사용할 2개의 숫자를 먼저 알려주어야 한다. 다음과 같이 연산을 수행할 대상(4, 2)을 객체에 지정할 수 있게 만들어 보자.

```
>>> a.setdata(4, 2)
```

위의 문장이 수행되려면 다음과 같이 소스 코드를 작성해야 한다.

```
>>> class FourCal:
...     def setdata(self, first, second):
...         self.first = first
...         self.second = second
...
>>>
```

이전에 만들었던 FourCal 클래스에서 pass라는 문장을 삭제하고 setdata라는 메서드를 만들었다.

일반적인 함수를 만들 때 우리는 다음과 같이 작성한다.

```
def sum(a, b):
    return a+b
```

즉, 입력값이 있고 수행하는 문장이 있다. 메서드도 마찬가지이다.

setdata 메서드를 다시 보면 아래와 같다.

```
def setdata(self, first, second): # ① 메서드의 입력 인수
    self.first = first           # ② 메서드의 수행문
    self.second = second         # ② 메서드의 수행문
```

① setdata 메서드의 입력 인수

입력 인수로 self, first, second라는 3개의 입력값을 받는다. 그런데 일반적인 함수와는 달리 메서드의 첫 번째 입력 인수는 특별한 의미를 갖는 self라는 변수이다.

다음의 예를 보면서 자세히 살펴보자.

```
>>> a = FourCal()
>>> a.setdata(4, 2)
```

위에서 보는 것처럼 a라는 객체를 만든 다음에 a.setdata(4, 2)처럼 하면 FourCal 클래스의 setdata 메서드가 호출되고 setdata 메서드의 첫 번째 인수에는 자동으로 a라는 객체가 입력으로 들어가게 된다.

즉, setdata의 입력 인수는 self, first, second로 총 3개이지만 a.setdata(4, 2)처럼 2개의 입력값만 주어도 a라는 객체가 setdata 메서드의 첫 번째 입력을 받는 변수인 self에 대입되게 된다.

self: 객체 a, first: 4, second: 2

파이썬 클래스에서 가장 헷갈리는 부분이 바로 이 부분이다. setdata라는 메서드는 입력 인수로 3개를 받는데 왜 a.setdata(4, 2)처럼 2개만 입력해도 실행이 되는가? 이 질문에 대한 답변을 여러분도 이제는 알았을 것이다.

[메서드의 또 다른 호출 방법]

잘 사용하지는 않지만 다음과 같이 메서드를 호출하는 것도 가능하다.

```
FourCal.setdata(a, 4, 2)
```

위와 같이 "클래스명.메서드" 형태로 호출할 때는 객체 a를 입력 인수로 꼭 넣어 주어야 한다. 반면에 앞에서 보았듯이 "객체.메서드" 형태로 호출할 때는 첫 번째 입력 인수(self)를 반드시 생략해야 한다.

그 다음으로 중요한 사항을 살펴보자.

② setdata 메서드의 수행문

setdata 메서드에는 수행할 문장이 2개 있다.

```
self.first = first
self.second = second
```

위 수행문이 뜻하는 바는 무엇일까? 입력 인수로 받은 first는 4이고 second는 2라는 것은 앞에서 이미 알았다. 그렇다면 위의 문장은 다음과 같이 바뀔 것이다.

```
self.first = 4
self.second = 2
```

여기서 중요한 것은 바로 self이다. self는 a.setdata(4, 2)처럼 호출했을 때 자동으로 첫 번째 인수로 들어오는 객체 a라고 했다. 그렇다면 self.first의 의미는 무엇이겠는가? 당연히 a.first가 될 것이다. 또한 self.second는 당연히 a.second가 될 것이다.

따라서 위의 두 문장을 풀어서 쓰면 다음과 같이 된다.


```
a.first = 4
a.second = 2
```

정말 이런지 직접 출력해서 확인해 보자.

```
>>> a = FourCal()
>>> a.setdata(4, 2)
>>> print(a.first)
4
>>> print(a.second)
2
```

b라는 객체를 하나 더 만들어 보자.

```
>>> b = FourCal()
>>> b.setdata(3, 7)
>>> print(b.first)
3
>>> print(a.first)
4
```

a와 b라는 객체는 모두 first라는 변수를 가지고 있지만 그 변수의 값은 각기 다르다. b 객체의 first 변수에 3이라는 값을 대입하더라도 a의 first 값이 3으로 변경되지는 않는다. a, b 객체의 first 변수는 고유의 저장 영역을 가지고 있는 객체 변수이기 때문이다.

객체 변수(예: self.first)는 그 객체의 고유한 값을 저장할 수 있는 공간이다. 객체 변수는 다른 객체들에 의해 영향받지 않고 독립적으로 그 값을 유지한다는 점을 꼭 기억하도록 하자. 클래스에 서는 이 부분을 이해하는 것이 가장 중요하다.

지금껏 완성된 클래스를 다시 작성해 보면 다음과 같다.

```
>>> class FourCal:
...     def setdata(self, first, second):
...         self.first = first
...         self.second = second
...
>>>
```

지금까지 살펴본 내용이 바로 위의 4줄을 설명하기 위한 것이었다. 위에서 설명한 것들이 이해가 되지 않는다면 다시 한 번 읽어 보기 바란다. 이 부분을 이해하지 못 하면 다음으로 넘어갈 수 없기 때문이다.

더하기 기능 만들기

자! 그럼 2개의 숫자값을 설정해 주었으니 2개의 숫자를 더하는 기능을 방금 만든 클래스에 추가해 보자. 우리는 다음과 같이 더하기 기능을 갖춘 클래스를 만들어야 한다.

```
>>> a = FourCal()
>>> a.setdata(4, 2)
```

```
>>> print(a.sum())
6
```

이를 가능하게 하기 위해 FourCal 클래스를 다음과 같이 만들었다.

```
>>> class FourCal:
...     def setdata(self, first, second):
...         self.first = first
...         self.second = second
...     def sum(self):
...         result = self.first + self.second
...         return result
...
>>>
```

새롭게 추가된 것은 sum이라는 메서드이다. 이 sum 메서드만 따로 떼어 내서 생각해 보자.

```
def sum(self):
    result = self.first + self.second
    return result
```

입력으로 받는 값은 self밖에 없고 돌려주는 값은 result이다. a.sum()처럼 수행하면 sum 메서드에 자동으로 객체 a가 첫 번째 입력 인수로 들어가게 된다는 것을 명심하자.

이번에는 돌려주는 값을 보자.

```
result = self.first + self.second
```

위의 내용은 아래와 같이 해석할 수 있다.

```
result = a.first + a.second
```

위의 내용은 a.setdata(4, 2)에서 a.first = 4, a.second = 2라고 이미 설정되었기 때문에 다시 다음과 같이 해석된다.

```
result = 4 + 2
```

따라서 아래와 같이 실행하면 6이 화면에 출력된다.

```
>>> print(a.sum())
6
```

여기까지 모두 이해한 독자라면 클래스에 대해 80% 이상을 안 것이다. 파이썬의 클래스는 그다지 어렵지 않다.

곱하기, 빼기, 나누기 기능 만들기

이번에는 곱하기, 빼기, 나누기 등을 할 수 있게 만들어 보자.

```
>>> class FourCal:
...     def setdata(self, first, second):
...         self.first = first
```

```

...     self.second = second
...     def sum(self):
...         result = self.first + self.second
...         return result
...     def mul(self):
...         result = self.first * self.second
...         return result
...     def sub(self):
...         result = self.first - self.second
...         return result
...     def div(self):
...         result = self.first / self.second
...         return result
...
>>>

```

mul, sub, div 모두 sum 메서드에서 배운 것과 동일한 방법이니 따로 설명하지는 않겠다.

정말로 모든 것이 제대로 동작하는지 확인해 보자.

```

>>> a = FourCal()
>>> b = FourCal()
>>> a.setdata(4, 2)
>>> b.setdata(3, 7)
>>> a.sum()
6
>>> a.mul()
8
>>> a.sub()
2
>>> a.div()
2
>>> b.sum()
10
>>> b.mul()
21
>>> b.sub()
-4
>>> b.div()
0

```

이상과 같이 우리가 목표로 했던 사칙연산 기능을 가진 클래스를 만들어 보았다.

"박씨네 집" 클래스 만들기

이번에는 전혀 다른 내용의 클래스를 한번 만들어 보자. 사칙연산 클래스보다 조금 더 재미있는 "박씨네 집"이라는 클래스를 만들어 보겠다. 먼저 이 클래스가 어떤 식으로 동작하게 할지 생각해 보자.

클래스 구상하기

1. 클래스 이름은 HousePark으로 하자. 다음처럼 pey라는 객체를 만든다.

```
>>> pey = HousePark()
```

2. pey.lastname을 출력하면 "박씨네 집"이라는 클래스에 걸맞게 "박"이라는 성을 출력하게 만들기 하자.

```
>>> print(pey.lastname)
박
```

3. 이름을 설정하면 pey.fullname이 성을 포함한 값을 가지도록 만든다.

```
>>> pey.setname("응용")
>>> print(pey.fullname)
박응용
```

4. 여행 가고 싶은 장소를 입력으로 주면 다음과 같이 출력해 주는 travel 메서드도 만들어 보자.

```
>>> pey.travel("부산")
박응용, 부산여행을 가다.
```

우선 여기까지만 만들어 보겠다. 1번부터 4번까지 순서대로 따라 해보면 어렵지 않을 것이다.

클래스 기능 만들기

1. 먼저 아무런 기능 없이 단순히 객체만 생성할 수 있는 클래스는 다음처럼 만들 수 있다.

```
>>>> class HousePark:
...     pass
...
>>>
```

2. 이렇게 클래스를 생성하면 pey = HousePark()처럼 입력해서 객체를 만들 수 있다. 이번에는 pey.lastname을 수행하면 "박"을 출력하기 위해 pass를 삭제한 후 다음처럼 입력해 보자.

```
>>> class HousePark:
...     lastname = "박"
...
>>>
```

lastname은 클래스 변수이다. 이 클래스 변수 lastname은 HousePark 클래스에 의해서 생성되는 객체 모두에 "박"이라는 값을 갖게 만든다.

다음의 예를 보자.

```
>>> pey = HousePark()
>>> pes = HousePark()
>>> print(pey.lastname)
박
```

```
>>> print(pes.lastname)
```

박

앞에서 봤듯이 HousePark 클래스에 의해서 생긴 객체 pey와 pes 모두 lastname이 "박"으로 설정되는 것을 확인할 수 있다.

(※ 클래스 변수는 HousePark.lastname처럼 "클래스명.변수명"으로도 사용할 수 있다.)

3. 이제 이름을 설정하고 print(pey.fullname)을 수행하면 성을 포함한 이름을 출력하도록 만들어 보자.

```
>>> class HousePark:
...     lastname = "박"
...     def setname(self, name):
...         self.fullname = self.lastname + name
...
>>>
```

우선 이름을 설정하기 위해 setname이라는 메서드를 만들었다. 이 메서드는 다음처럼 사용할 수 있다.

```
>>> pey = HousePark()
>>> pey.setname("응용")
```

setname 메서드에 "응용"이라는 값을 인수로 주면 self.fullname에는 결국 "박" + "응용"이라는 값이 대입된다.

"응용"을 입력으로 주었을 때 "박응용"이 생성되는 과정을 살펴보자. 다음은 성을 포함한 이름을 만드는 문장이다.

```
self.fullname = self.lastname + name
```

setname 메서드에서 두 번째 입력값 name은 "응용"이므로 위의 문장은 다음과 같이 바뀐다.

```
self.fullname = self.lastname + "응용"
```

이 문장에서 self는 setname 메서드의 첫 번째 입력으로 들어오는 pey라는 객체이기 때문에 다시 다음과 같이 바뀐다.

```
pey.fullname = pey.lastname + "응용"
```

pey.lastname은 클래스 변수로 항상 "박"이라는 값을 갖기 때문에 다음과 같이 바뀐다.

```
pey.fullname = "박" + "응용"
```

따라서 pey.fullname을 출력하면 다음과 같은 결과를 얻을 수 있다.

```
>>> print(pey.fullname)
```

박응용

4. 이제 우리가 만들려고 했던 클래스의 기능 중 단 한 가지만 남았다. 입력받은 장소로 박응용이 여행을 간다고 출력해 주는 travel 메서드를 HousePark 클래스에 구현해 보자.

```
>>> class HousePark:
...     lastname = "박"
...     def setname(self, name):
...         self.fullname = self.lastname + name
...     def travel(self, where):
...         print("%s, %s여행을 가다." % (self.fullname, where))
...
>>>
```

travel 메서드는 입력값으로 객체(self)와 장소(where)를 받는다. 그리고 해당 값들을 문자열 포매팅 연산자(%)를 이용하여 문자열에 삽입한 후 출력한다.

위 클래스는 다음과 같이 사용할 수 있다.

```
>>> pey = HousePark()
>>> pey.setname("응용")
>>> pey.travel("부산")
박응용, 부산여행을 가다.
```

위의 과정을 travel 메서드의 입장에서 살펴보면 다음과 같다.

우선 travel 메서드의 입력 변수인 self와 where은 다음과 같을 것이다.

```
self: pey
where : "부산"
```

따라서 self.fullname은 pey.fullname이 될 것이고, pey.fullname은 pey.setname("응용")에 의해서 만들어진 "박응용"이 될 것이다. 따라서 pey.travel("부산")처럼 입력하면 위의 예처럼 출력되는 것이다.

초깃값 설정하기

우리는 위에서 HousePark이라는 클래스를 이용해서 객체를 만들었다. 그리고 이 객체에 setname 메서드를 이용해서 사람 이름을 설정해 주는 방식을 사용했다.

그런데 위에서 만든 메서드를 다음과 같이 실행하면 어떻게 될까?

```
>>> pey = HousePark()
>>> pey.travel("부산")
```

다음과 같은 오류가 발생한다.

```
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
File "<stdin>", line 6, in travel
```

AttributeError: 'HousePark' object has no attribute 'fullname'

오류가 발생한 이유는 travel 메서드가 self.fullname이라는 변수를 필요로 한다는 데서 찾을 수 있다. 앞에서 말했듯이 이름을 설정하는 self.fullname 변수는 setname이라는 메서드에 의해서 생성된다. 그런데 위의 경우 setname 메서드가 실행되지 않아 오류가 발생하는 것이다.

클래스를 설계할 때 이렇게 오류가 발생할 수 있는 상황을 차단하지 못 하면 좋은 클래스라고 보기 어렵다. 따라서 이런 오류가 발생하는 것을 방지하기 위해 pey라는 객체를 만드는 순간 setname 메서드가 동작하게 한다면 상당히 편리할 것이다. 이러한 생각으로 나오게 된 것이 바로 `__init__`이라는 메서드이다.

`__init__` 메서드로 초깃값을 설정한다

자, 그렇다면 HousePark 클래스를 다음과 같이 바꾸어 보자.

```
>>> class HousePark:
...     lastname = "박"
...     def __init__(self, name):
...         self.fullname = self.lastname + name
...     def travel(self, where):
...         print("%s, %s여행을 가다." % (self.fullname, where))
...
>>>
```

setname 메서드 이름이 `__init__`으로 바뀌기만 했다.

이것이 어떤 차이를 가져오는지 알아보자. 다음처럼 입력 해 보자.

```
>>> pey = HousePark()
TypeError: __init__() takes exactly 2 arguments (1 given)
```

위와 같은 오류 메시지를 볼 수 있을 것이다. 메시지를 읽어 보면 알 수 있듯이 pey = HousePark()이라고 입력하는 순간 `__init__` 메서드가 호출된다. `__init__` 메서드는 2개의 입력값 (self, name)이 필요한데 1개의 입력값만 받았기 때문에 오류가 발생한 것이다. 여기서 `__init__` 메서드가 받은 입력값 1개는 입력 인수 self를 통해 받는 pey라는 객체이다.

그렇다면 객체를 만들 때 어떻게 `__init__` 메서드에 입력값을 2개 줄 수 있을까? 그 방법은 다음과 같다.

```
>>> pey = HousePark("응용")
```

이전에 보았던 setname 메서드를 사용하는 방법인 pey.setname("응용")과 아주 비슷하다. 다만 객체를 생성하는 순간에 입력값으로 "응용"이라는 값을 주는 점만 다르다.

이렇게 `__init__` 메서드를 이용하면 객체를 만드는 동시에 초깃값을 줄 수 있기 때문에 훨씬 편리하다.

(※ `__init__` 메서드는 생성자(Constructor)라고도 한다.)

이제 다음과 같이 입력해 보자. 오류 없이 잘 실행되는 것을 확인할 수 있을 것이다.

```
>>> pey = HousePark("응용")
>>> pey.travel("태국")
박응용, 태국여행을 가다.
```

클래스의 상속

상속(Inheritance)이란 "물려받다"라는 뜻으로, "재산을 상속받다"라고 할 때의 상속과 같은 의미이다. 클래스에도 이런 개념을 적용할 수가 있다. 어떤 클래스를 만들 때 다른 클래스의 기능을 물려받을 수 있게 만드는 것이다. 우리는 "박씨네 집"이라는 HousePark 클래스를 만들었다. 이번엔 "김씨네 집"이라는 HouseKim 클래스를 만들어 보자.

상속의 개념을 이용하면 다음과 같이 간단하게 HouseKim 클래스를 만들 수 있다. 다음은 HouseKim이라는 클래스가 HousePark 클래스를 상속받는 예제이다. HousePark 클래스는 이미 만들어 놓았다고 가정한다.

```
>>> class HouseKim(HousePark):
...     lastname = "김"
...
>>>
```

성(lastname)을 "김"으로 고쳤을 뿐 그 외에는 아무것도 추가하지 않았다.

앞에서 봤듯이 클래스명 뒤 괄호 안에 다른 클래스명을 넣어 주면 상속을 받게 된다.

class 상속받을 클래스명(상속할 클래스명)

이제 다음과 같이 따라 해보자.

```
>>> juliet = HouseKim("줄리엣")
>>> juliet.travel("독도")
김줄리엣, 독도여행을 가다.
```

HouseKim 클래스는 HousePark 클래스의 모든 기능을 그대로 상속받았기 때문에 `__init__` 메서드와 `travel` 메서드를 구현하지 않더라도 HousePark 클래스와 완전히 동일하게 동작하는 것을 볼 수 있다.

재미있지 않은가? 상속의 개념을 이용하면 독자는 "양씨네 집", "이씨네 집" 등을 쉽게 만들 수 있을 것이다.

메서드 오버라이딩

상속의 개념 중 한 가지 더 알아야 할 것이 있다. 클래스를 만들다 보면 상속받을 대상인 클래스의 메서드와 이름은 같지만 그 행동을 다르게 해야 할 때가 있다. 이럴 때는 어떻게 해야 할까?

이전 예제에서 "김씨네 집" 클래스는 "박씨네 집" 클래스를 상속받았다. 우리는 여기서 "김씨네 집" 클래스가 상속받은 travel 메서드를 "박씨네 집" 클래스의 travel 메서드와 다르게 동작하도록 만들어 볼 것이다.

```
>>> juliet = HouseKim("줄리엣")
>>> juliet.travel("독도")
김줄리엣은 독도로 여행합니다.
```

(※ HouseKim 클래스의 travel 메서드는 HousePark클래스와 출력문자열이 서로 다르다.)

앞의 예처럼 행동할 수 있도록 HouseKim 클래스를 만들어 보자.

```
>>> class HouseKim(HousePark):
...     lastname = "김"
...     def travel(self, where):
...         print("%s은 %s로 여행합니다." % (self.fullname, where))
...
>>>
```

travel 메서드를 다르게 설정하고 싶으면 동일한 이름의 travel 메서드를 HouseKim 클래스 내에서 다시 구현하면 된다. 이렇게 메서드 이름을 동일하게 다시 구현하는 것을 메서드 오버라이딩(Overriding)이라고 한다.

연산자 오버로딩

연산자 오버로딩(Overloading)이란 연산자(+, -, *, /,...)를 객체끼리 사용할 수 있게 하는 기법으로, 연산자 오버로딩을 사용하면 다음과 같이 동작하도록 만들 수 있다.

```
>>> pey = HousePark("응용")
>>> juliet = HouseKim("줄리엣")
>>> pey + juliet
박응용, 김줄리엣 결혼했네
>>>
```

HousePark클래스를 다음과 같이 수정해 보자.

```
# house.py
```

```
class HousePark:
    lastname = "박"
    def __init__(self, name):
        self.fullname = self.lastname + name
    def travel(self, where):
        print("%s, %s여행을 가다." % (self.fullname, where))
```

```

def __add__(self, other):
    print("%s, %s 결혼했네" % (self.fullname, other.fullname))

class HouseKim(HousePark):
    lastname = "김"
    def travel(self, where):
        print("%s은 %s로 여행합니다." % (self.fullname, where))

pey = HousePark("응용")
juliet = HouseKim("줄리엣")
pey + juliet

```

위의 프로그램을 실행시키면 다음과 같은 결과를 얻을 수 있다.

박응용, 김줄리엣 결혼했네

먼저 위 프로그램의 실행 부분인 다음을 살펴보자.

```

pey = HousePark("응용")
juliet = HouseKim("줄리엣")
pey + juliet

```

먼저 pey = HousePark("응용")으로 pey라는 객체를 만들고 juliet이라는 객체도 생성한다.

자! 이제 다음 문장을 살펴보자.

```
pey + juliet
```

더하기 기호인 +를 이용해서 객체끼리 더하려고 한다. 이렇게 + 연산자를 객체에 사용하게 되면 HousePark 클래스의 __add__ 라는 메서드가 자동으로 호출된다.

HousePark 클래스에서 __add__ 메서드가 선언된 부분을 보면 다음과 같다.

```

def __add__(self, other):
    print("%s, %s 결혼했네" % (self.fullname, other.fullname))

```

pey + juliet처럼 호출되면 __add__(self, other) 메서드의 self에는 pey 객체가 전달되고 other는 juliet 객체가 전달된다. 따라서 "박응용, 김줄리엣 결혼했네"라는 문자열이 출력된다.

"박씨네 집" 클래스 완성하기

자, 이젠 지금껏 만들어 본 클래스로 이야기를 만들어 보자. 스토리는 다음과 같다.

박응용은 독도에 놀러 가고
 김줄리엣도 독도에 놀러 간다.
 둘은 결혼하게 된다.
 그러다가 헤어지게 된다.

참으로 슬픈 이야기이지만 연산자 오버로딩을 공부하기에는 좋은 스토리이다.

```
pey = HousePark("응용")
juliet = HouseKim("줄리엣")
pey.travel("독도")
juliet.travel("독도")
pey + juliet
pey - juliet
```

위와 같이 동작시키면 결과값이 다음처럼 나오게 만드는 클래스를 작성하는 것이 우리의 목표이다.

박응용 독도여행을 가다.
김줄리엣은 독도로 여행합니다.
박응용, 김줄리엣 결혼했네
박응용, 김줄리엣 헤어졌네

pey - juliet을 수행하기 위해 `__sub__` 이라는 빼기(-) 연산자가 사용되었을 때 호출되는 메서드를 만들어 주어야 한다. 이것도 역시 연산자 오버로딩을 기법을 사용한다.

(※ +, - 이외에 다른 연산자들의 오버로딩도 가능하다.(예: 곱하기(*)는 `__mul__`, 나누기(/)는 `__truediv__` 등))

자, 최종적으로 만들어진 "박씨네 집" 클래스를 공개한다.

```
class HousePark:
    lastname = "박"
    def __init__(self, name):
        self.fullname = self.lastname + name
    def travel(self, where):
        print("%s, %s여행을 가다." % (self.fullname, where))
    def __add__(self, other):
        print("%s, %s 결혼했네" % (self.fullname, other.fullname))
    def __sub__(self, other):
        print("%s, %s 헤어졌네" % (self.fullname, other.fullname))
```

```
class HouseKim(HousePark):
    lastname = "김"
    def travel(self, where):
        print("%s은 %s로 여행합니다." % (self.fullname, where))
```

```
pey = HousePark("응용")
juliet = HouseKim("줄리엣")
pey.travel("독도")
juliet.travel("독도")
pey + juliet
pey - juliet
```

결과값은 예상한 대로 다음처럼 나올 것이다.

박응용 독도여행을 가다.

김줄리엣은 독도로 여행합니다.

박응용, 김줄리엣 결혼했네

박응용, 김줄리엣 헤어졌네